

Implementazione dell'algoritmo Flooding con FIFO Linux

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 15

Indice

1	Introduzione al problema	2
2	Stato dell'arte	2
3	Metodologia usata	3
3.1	Rappresentazione del grafo	3
3.1.1	Grafo	4
3.2	Strutture dati principali	4
3.3	Creazione dei processi	5
3.4	Gestione delle FIFO	5
3.5	Funzione msg	6
3.6	Funzione stf	6
4	Risultati sperimentali	6
5	Conclusione e sviluppi futuri	7

1 Introduzione al problema

L'obiettivo dell'esercizio è realizzare l'algoritmo distribuito di *Flooding* utilizzando il linguaggio C e le FIFO di Linux come meccanismo di comunicazione tra processi.

Il problema richiede di modellare una rete di comunicazione composta da più agenti collegati tra loro tramite canali direzionali. Ogni agente è rappresentato da un processo separato, mentre ogni canale di comunicazione è rappresentato da una FIFO Linux. In questo modo, la struttura del programma riproduce il comportamento di una rete distribuita, nella quale ogni nodo possiede uno stato locale e comunica solo con i propri vicini.

L'algoritmo di Flooding prevede che un nodo iniziale, nel nostro caso il nodo 1, possieda un'informazione detta *token*. Tale informazione viene inviata ai nodi vicini e successivamente inoltrata dagli altri nodi della rete.

Lo scopo è fare in modo che tutti i nodi raggiungibili ricevano il token, evitando però che il messaggio venga ritrasmesso indefinitamente.

Nel programma ogni agente mantiene uno stato locale composto da tre informazioni principali:

- **parent**, cioè il nodo da cui l'agente ha ricevuto il token per la prima volta;
- **data**, cioè il valore del token ricevuto;
- **snd_flag**, cioè un flag che indica se l'agente deve ancora inviare il token ai propri vicini.

Alla fine dell'esecuzione, i valori di **parent** permettono di ricostruire l'albero di flooding generato dall'algoritmo.

2 Stato dell'arte

Nei sistemi distribuiti, il flooding è una tecnica semplice e molto usata per propagare informazioni all'interno di una rete. L'idea alla base è che ogni nodo, quando riceve per la prima volta un messaggio, lo inoltra ai propri vicini.

In questo modo, il messaggio può raggiungere progressivamente tutti i nodi connessi.

Il vantaggio principale del flooding è la sua semplicità: non è necessario conoscere globalmente la rete, ma ogni nodo deve conoscere solo i propri vicini.

Tuttavia, se non viene controllato correttamente, il flooding può generare messaggi ridondanti, soprattutto in presenza di cicli. Per evitare questo problema, ogni nodo deve memorizzare se ha già ricevuto il messaggio e deve evitare di ritrasmetterlo più volte.

Nel contesto dell'esercizio, la comunicazione tra agenti viene realizzata tramite FIFO Linux, dette anche *named pipe*. Una FIFO è un file speciale che permette a processi diversi di comunicare seguendo una politica *First In, First Out*: i dati letti sono restituiti nello stesso ordine in cui sono stati scritti.

Rispetto alle pipe anonime, le FIFO hanno un nome nel filesystem e possono quindi essere aperte da processi indipendenti. Nel progetto realizzato, ogni arco direzionale del grafo corrisponde a una FIFO distinta.

Questa scelta consente di modellare in modo chiaro i canali della rete distribuita.

3 Metodologia usata

Il progetto è stato organizzato in modo modulare, mantenendo però una struttura semplice. Tutti i file si trovano nella cartella `codice`. La struttura del progetto è la seguente:

```
codice/  
|-- Makefile  
|-- common.h  
|-- graph.c  
|-- flooding.c  
|-- main.c  
|-- graph.txt
```

Il file `Makefile` automatizza la compilazione del progetto. I file sorgenti producono l'eseguibile finale `flooding_fifo`.

3.1 Rappresentazione del grafo

Il grafo della rete non è scritto direttamente nel codice, ma viene letto da un file esterno chiamato `graph.txt`. Questa scelta rende il programma più modulare, perché permette di modificare la topologia della rete senza cambiare il codice sorgente.

Il file `graph.txt` usato nel progetto è il seguente:

```
6 12  
5 6  
6 5  
6 4  
5 1  
1 4  
1 3  
1 2  
3 4  
2 4  
2 3  
2 5  
3 5
```

La prima riga indica il numero di nodi e il numero di archi direzionali. Ogni riga successiva rappresenta un arco del tipo:

`sorgente destinazione`

3.1.1 Grafo

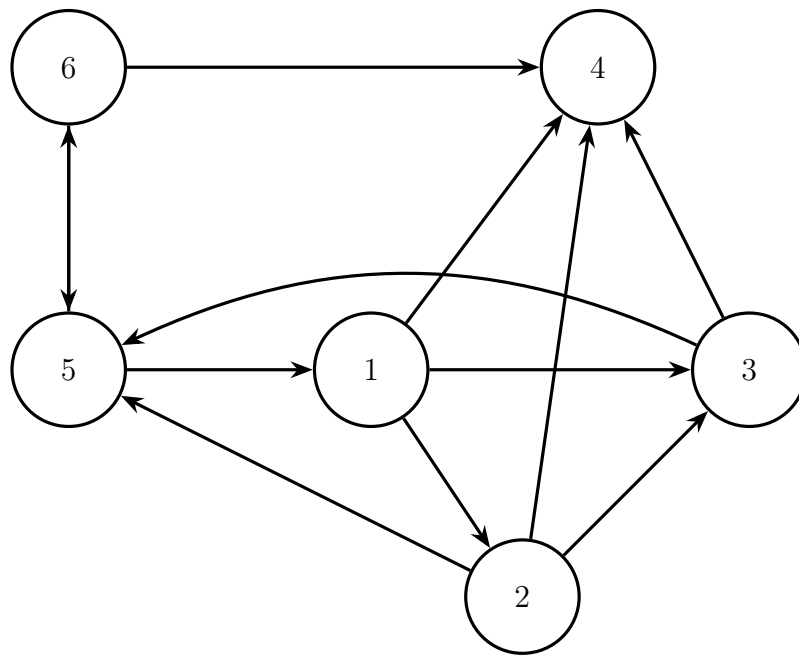


Figura 1: Rappresentazione del grafo di comunicazione descritto nel file `graph.txt`.

Il file `graph.c` contiene la funzione `read_graph`, che legge il file del grafo e riempie una struttura dati di tipo `Graph`. Tale struttura contiene:

- il numero di nodi;
- il numero di archi;
- la lista completa degli archi;
- i vicini uscenti di ogni nodo;
- i vicini entranti di ogni nodo.

3.2 Strutture dati principali

Le strutture dati condivise tra i vari file sono definite in `common.h`.

La struttura `Edge` rappresenta un arco direzionale del grafo:

```
typedef struct {  
    int src; //nodo sorgente  
    int dst; //nodo destinazione  
} Edge;
```

La struttura `Message` rappresenta un messaggio inviato tramite FIFO:

```
typedef struct {  
    int sender; //mittente del messaggio  
    int is_null; //se e' nullo o contiene token  
    int data; //il valore del token  
} Message;
```

Infine, la struttura `State` rappresenta lo stato locale di un agente:

```
typedef struct {
    int parent; //da quale nodo si riceve il token la 1 volta
    int has_data; //se l'agente ha il token
    int data; //valore del token
    bool snd_flag; //se il token deve essere inoltrato
} State;
```

3.3 Creazione dei processi

Nel file `main.c`, il processo principale legge il grafo, crea le FIFO e genera un processo figlio per ogni nodo della rete tramite la chiamata di sistema `fork()`.

La parte principale è la seguente:

```
for(int id = 1; id <= g.n_nodes; id++) {
    pid_t pid = fork();

    if(pid == -1) {
        die("fork");
    }

    if(pid == 0) {
        agent_process(id, &g, run_id, rounds, token);
    }
}
```

Ogni processo figlio esegue la funzione `agent_process`, che rappresenta il comportamento di un agente della rete. Il processo padre, invece, aspetta la terminazione di tutti i figli tramite `wait()` e, al termine dell'esecuzione, rimuove le FIFO create.

3.4 Gestione delle FIFO

La gestione delle FIFO è implementata nel file `flooding.c`. Per ogni arco del grafo viene creata una FIFO con un nome univoco. Il nome contiene anche il PID del processo principale, in modo da evitare conflitti tra esecuzioni diverse del programma.

La funzione che costruisce il nome della FIFO è:

```
static void fifo_name(char* buffer, size_t size, pid_t run_id, int src,
int dst) {
    snprintf(buffer, size, "/tmp/lab15_fifo_%ld_%d_%d",
              (long)run_id, src, dst);
}
```

Le FIFO vengono create con `mkfifo()` e rimosse con `unlink()`.

Prima della creazione, il programma richiama `unlink()` per eliminare eventuali FIFO rimaste da esecuzioni precedenti terminate in modo anomalo.

Le FIFO sono utilizzate con operazioni di lettura e scrittura bloccanti.

L'apertura delle FIFO avviene con il flag `O_RDWR`. Questa scelta permette di evitare il blocco iniziale della `open()` quando un processo apre un canale solo in lettura o solo in scrittura senza che l'altro estremo sia già aperto.

3.5 Funzione msg

La funzione `msg_function` implementa la funzione `msg` dell'algoritmo di flooding. Essa decide quale messaggio inviare a un determinato vicino.

La regola usata è la seguente:

- se l'agente possiede il token;
- se `snd_flag` è vero;
- e se il destinatario non è il `parent`;

allora l'agente invia il token.

In caso contrario invia un messaggio nullo.

Questa funzione evita che un nodo invii nuovamente il token al nodo da cui lo ha ricevuto.

3.6 Funzione stf

La funzione `stf_function` implementa la funzione di transizione di stato. Essa riceve lo stato corrente dell'agente e l'insieme dei messaggi ricevuti nel round, restituendo il nuovo stato.

I casi gestiti sono tre:

1. l'agente non ha ancora il token e riceve solo messaggi nulli;
2. l'agente non ha ancora il token e riceve almeno un messaggio non nullo;
3. l'agente possiede già il token.

Nel primo caso l'agente rimane senza token. Nel secondo caso salva il token, imposta il mittente come `parent` e abilita `snd_flag`. Se più messaggi non nulli arrivano nello stesso round, viene scelto come `parent` il mittente con identificativo più piccolo. Nel terzo caso, l'agente mantiene `parent` e `data`, ma disattiva `snd_flag`, evitando ulteriori ritrasmissioni.

4 Risultati sperimentali

Il programma è stato compilato tramite il comando:

```
make
```

ed eseguito con:

```
make run
```

oppure scrivendo:

```
./flooding_fifo graph.txt 6 42
```

In questa esecuzione, il programma utilizza il file `graph.txt`, esegue 6 round e propaga il token di valore 42.

Durante l'esecuzione, i processi stampano a video i messaggi inviati, i messaggi ricevuti e i cambiamenti di stato. Poiché gli agenti sono processi separati che eseguono contemporaneamente, l'ordine delle stampe può risultare interlacciato.

Per verificare più facilmente il risultato finale, è possibile filtrare l'output con:

```
./flooding_fifo graph.txt 6 42 | grep "^\[fine\]" | sort
```

L'output finale ottenuto è:

```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho15/codice$ ./flooding_fifo graph.txt 6 42 | grep "^\[fine\]" | sort
[fine] agente 1: parent = 1, data = TOKEN, radice dell'albero
[fine] agente 2: parent = 1, data = TOKEN, 1 -> 2
[fine] agente 3: parent = 1, data = TOKEN, 1 -> 3
[fine] agente 4: parent = 1, data = TOKEN, 1 -> 4
[fine] agente 5: parent = 2, data = TOKEN, 2 -> 5
[fine] agente 6: parent = 5, data = TOKEN, 5 -> 6
```

Da questo output si ricava l'albero di flooding finale:

```
1 -> 2
1 -> 3
1 -> 4
2 -> 5
5 -> 6
```

Il nodo 1 è la radice dell'albero, poiché è il nodo iniziale. Tutti gli altri nodi hanno ricevuto correttamente il token e hanno memorizzato il nodo da cui lo hanno ricevuto per la prima volta.

Il risultato conferma il corretto funzionamento dell'algoritmo: il token viene propagato a tutti i nodi della rete e i valori di **parent** permettono di ricostruire l'albero generato dal flooding.

5 Conclusione e sviluppi futuri

Il progetto realizzato implementa correttamente l'algoritmo di Flooding utilizzando processi e FIFO Linux. Ogni agente è rappresentato da un processo separato, mentre ogni canale direzionale del grafo è rappresentato da una FIFO. Il grafo viene letto da un file esterno, rendendo il programma più modulare e più semplice da modificare.

Le funzioni principali dell'algoritmo, **msg** e **stf**, sono state implementate separando la logica di invio dalla logica di aggiornamento dello stato. La funzione **msg_function** decide se inviare il token o un messaggio nullo, mentre la funzione **stf_function** aggiorna lo stato locale dell'agente in base ai messaggi ricevuti nel round.

L'esecuzione ha mostrato che tutti i nodi ricevono il token e che i valori finali di **parent** producono l'albero di flooding atteso.

Come possibili sviluppi futuri si potrebbero considerare:

- il supporto a grafi più grandi o caricati da formati più complessi;
- una gestione più dettagliata degli errori di comunicazione tra processi;

- una visualizzazione grafica dell'albero risultante;
- il confronto tra implementazione con processi e implementazione con thread.